Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ПРАКТИЧЕСКАЯ РАБОТА №2

По дисциплине Контейнеризация и оркестрация приложений

Тема работы Создание и оптимизация Docker-образа веб-приложения

Обучающийся Рекун Ирина Данииловна

Факультет Факультет прикладной информатики

Группа K3320

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся			Рекун И.Д.
	(дата)	(подпись)	(Ф.И.О.)
Руководитель			Аминов Н.С.
	(дата)	(подпись)	(Ф.И.О.)

Санкт-Петербург 2026

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ 3
1 ПЕРВЫЙ РАБОЧИЙ ОБРАЗ 4
1.1 Базовый Dockerfile и запуск контейнера 4
1.2 Проверка HTTP-маршрутов 5
2 ОПТИМИЗАЦИЯ DOCKER-ОБРАЗА 6
2.1 Сборка версии v2 и сравнение размеров 6
2.2 Многостадийная сборка 7
3 ИСКЛЮЧЕНИЕ ФАЙЛОВ ИЗ КОНТЕКСТА СБОРКИ 8
4 ПРОВЕРКА СОСТОЯНИЯ КОНТЕЙНЕРА 9
5 ТЕГИРОВАНИЕ И ФИНАЛЬНАЯ ПРОВЕРКА 10
ЗАКЛЮЧЕНИЕ 11

ВВЕДЕНИЕ

В данной практической работе рассматривается подготовка Docker-образа для учебного веб-приложения Personal Assistant на Python и FastAPI. Работа выполнялась в терминале Ubuntu, запущенной через WSL на Windows 11, с использованием Docker Desktop. В ходе работы создан базовый образ, проверено взаимодействие с приложением по HTTP, выполнена оптимизация зависимостей, настроены исключения файлов и контроль работоспособности контейнера.

Цель работы — научиться самостоятельно создавать и оптимизировать Docker-образы, запускать контейнер с персональными параметрами, проверять доступность сервиса и фиксировать воспроизводимый результат. Дополнительно исследована многостадийная сборка, позволяющая разделить подготовку зависимостей и формирование окружения запуска.

Персональные значения представлены в таблице 1. Порт вычислен по номеру ИСУ: 5430 + 28 = 5458. Для имён контейнеров и образов использован префикс rekun.

Таблица 1 – Персональные параметры

Параметр	Значение
Номер ИСУ	467228
Персональный порт	5458
Персональный префикс	rekun
Секретный ключ	secret28rekun

1 ПЕРВЫЙ РАБОЧИЙ ОБРАЗ

1.1 Базовый Dockerfile и запуск контейнера

На первом этапе был подготовлен Dockerfile на основе python:3.11-slim. В нём указаны автор и версия образа, рабочая директория /app, последовательность установки зависимостей и команда запуска веб-сервера Uvicorn. Сначала в образ копируется requirements.txt, устанавливаются библиотеки Python, а затем копируется каталог приложения. Такой порядок важен для повторного использования кеша слоёв: изменение программного кода не требует переустановки неизменившихся зависимостей.

Базовый Dockerfile сохранён отдельным коммитом в персональном репозитории GitHub (рисунок 1). Базовая версия образа была собрана под тегом v1, её размер составил 188 МБ.

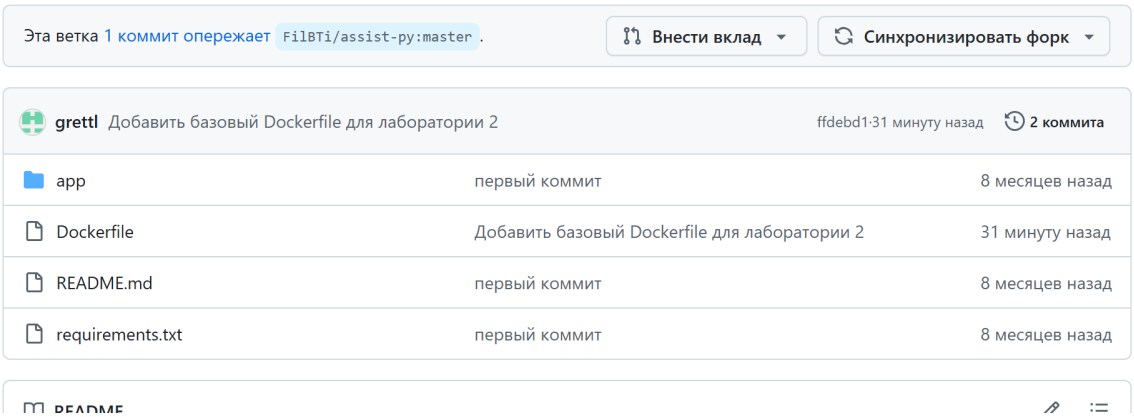


Рисунок 1 – Базовый Dockerfile в GitHub

Контейнер запускался в фоновом режиме с именем rekun-assistant-1. Публикация порта 5458:5458 связывает порт компьютера с портом приложения внутри контейнера. Переменные APP_OWNER и APP_SECRET передаются во время запуска, поэтому для персонализации не требуется менять исходный код. Инструкция EXPOSE в Dockerfile только обозначает предполагаемый внутренний порт; фактическую публикацию выполняет параметр -p при запуске.

1.2 Проверка HTTP-маршрутов

После запуска проверена работа приложения запросами curl. На рисунке 2 видно, что маршрут главной страницы возвращает приветствие для rekun; маршрут /me показывает владельца и начало секретного ключа. Затем был отправлен запрос POST для создания заметки, а повторный запрос GET к /notes вернул список и количество записей. Получение флага LAB2_SUCCESS_REKUN при обращении к /secret подтверждает совпадение переданного ключа со значением переменной окружения.

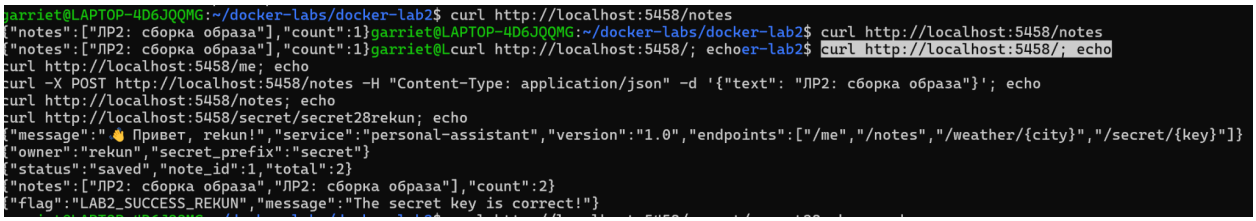


Рисунок 2 – Проверка маршрутов приложения

Таким образом, проверка охватывает не только запуск процесса, но и доступность HTTP-интерфейса, обработку JSON-запроса и использование настроек окружения. В тесте заметок параметры запроса определяли метод POST, тип передаваемых данных и содержимое записи. Наличие успешного ответа от приложения подтверждает корректную публикацию порта 5458.

2 ОПТИМИЗАЦИЯ DOCKER-ОБРАЗА

2.1 Сборка версии v2 и сравнение размеров

Для оптимизации был создан файл Dockerfile.optimized. В его первоначальном варианте сохранено отдельное копирование файла зависимостей и использован параметр --no-cache-dir при установке пакетов. Он запрещает pip сохранять свой кеш скачанных пакетов в слое образа. Указанный вариант файла, представленный на рисунке 3, применялся именно при сборке версии v2; позднее этот же файл был дополнен Healthcheck и переработан в многостадийный.

```
FROM python:3.11-slim
LABEL maintainer="rekun@lab.local"
LABEL version="2.0"

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ ./app/

EXPOSE 5458

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "5458"]
```

Рисунок 3 – Оптимизированный Dockerfile

На рисунке 4 приведён результат сравнения двух версий. Базовый образ v1 занимает 188 МБ, а оптимизированный v2 — 175 МБ. Уменьшение составило 13 МБ, или приблизительно 6,9%. Размер образа уменьшился за счёт исключения кеша загруженных Python-пакетов.

```
garriet@LAPTOP-4D6JQQMG:~/docker-labs/docker-lab2$ docker images | grep rekun-assistant
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
rekun-assistant:v1                  6e40fd3f923e        188MB             0B   U
rekun-assistant:v2                  54f5106f91b1        175MB             0B
garriet@LAPTOP-4D6JQQMG:~/docker-labs/docker-lab2$
```

Рисунок 4 – Сравнение размеров образов v1 и v2

Отдельное копирование requirements.txt ускоряет повторную сборку, потому что Docker повторно использует слой установки пакетов, если список зависимостей не изменился. Копирование кода после установки пакетов позволяет не пересобирать этот слой при каждом изменении логики приложения. Отключение кеша pip влияет преимущественно на размер образа, а повторное использование слоёв Docker — на время сборки.

2.2 Многостадийная сборка

В Dockerfile.optimized созданы стадии builder и runner. В первой зависимости устанавливаются в каталог /install, затем во вторую копируются библиотеки и код приложения (рисунок 5). Команда запуска приведена на рисунке 6.

```
garriet@LAPTOP-4D6JQQMG:~/docker-labs/docker-lab2$ cat Dockerfile.optimized
# Первая стадия: установка Python-зависимостей
FROM python:3.11-slim AS builder

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# Вторая стадия: окружение для запуска приложения
FROM python:3.11-slim AS runner

LABEL maintainer="rekun@lab.local"
LABEL version="2.1"

WORKDIR /app

# Установка curl для проверки состояния приложения
RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*

# Копирование библиотек из первой стадии и кода приложения
COPY --from=builder /install /usr/local
COPY app/ ./app/

EXPOSE 5458

# Проверка работоспособности приложения
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD curl -f http://localhost:5458/health || exit 1
```

Рисунок 5 – Настройка многостадийной сборки

```
+
 CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "5458"]
garriet@LAPTOP-4D6JQQMG:~/docker-labs/docker-lab2$
```

Рисунок 6 – Команда запуска приложения

Таблица 2 показывает, как менялся размер образа. Версия v1 занимает 188 МБ. После отключения кеша pip версия v2 уменьшилась до 175 МБ. Добавление .dockerignore в v3 не изменило размер исходного образа. Итоговая версия final занимает 178 МБ: она больше v2, поскольку содержит curl для проверки работы приложения.

Таблица 2 – Сравнение размеров полученных образов

Версия образа	Размер
v1 — исходный образ	188 МБ
v2 — без кеша pip	175 МБ
v3 — с файлом .dockerignore	188 МБ
final — многостадийная сборка	178 МБ

3 ИСКЛЮЧЕНИЕ ФАЙЛОВ ИЗ КОНТЕКСТА СБОРКИ

Для управления содержимым контекста создан файл .dockerignore. На рисунке 7 видны шаблоны, исключающие кеш Python, скомпилированные файлы, виртуальные окружения, Git-метаданные, журналы и вспомогательные документы. Эти данные не нужны запущенному FastAPI-приложению и не должны без необходимости передаваться в процесс сборки.

```
garriet@LAPTOP-4D6JQQMG:~/docker-labs/docker-lab2$ cat .dockerignore
__pycache__/
*.pyc
*.pyo
.git
.gitignore
.venv/
venv/
.env
*.log
Dockerfile*
README.md
```

Рисунок 7 – Содержимое .dockerignore

После добавления файла была повторно собрана базовая версия с тегом v3. Её размер остался равным 188 МБ. Это ожидаемо: .dockerignore меняет содержимое передаваемого контекста, но не уменьшает автоматически каждый слой готового образа. В данном случае сборка выполнялась по базовому Dockerfile, поэтому версия v3 не наследовала оптимизацию установки pip из v2.

Без исключений в контекст могли попасть каталоги .git и venv, временные файлы, локальные журналы и файлы настроек. История Git может содержать удалённые ранее конфигурации и чувствительные сведения. При копировании чрезмерно широкого набора файлов они способны оказаться в слоях образа; даже если их удалят позже, сведения могут сохраниться в предыдущем слое. Поэтому исключения уменьшают лишнюю передачу файлов и помогают снизить риск раскрытия данных.

4 ПРОВЕРКА СОСТОЯНИЯ КОНТЕЙНЕРА

В финальный Dockerfile добавлена инструкция HEALTHCHECK, выполняющая HTTP-запрос к маршруту /health. Интервал между проверками установлен в 30 секунд, тайм-аут отдельной проверки — 5 секунд, начальный период — 10 секунд, допустимое количество последовательных неудачных проверок — три. Приложение отвечает на маршрут /health JSON-сообщением о своём состоянии.

После запуска была проверена работоспособность финального контейнера. В выводе docker ps отображается статус healthy (рисунок 8), который подтверждает успешное выполнение автоматической HTTP-проверки. Отметка Up показывает, что основной процесс контейнера работает, а healthy дополнительно подтверждает доступность проверяемого маршрута приложения.

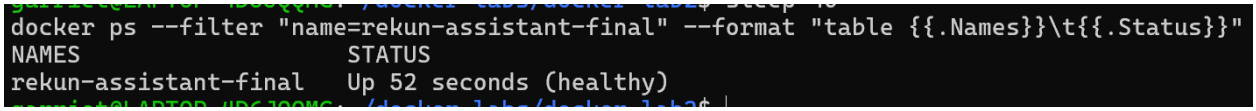


Рисунок 8 – Проверка состояния контейнера

Docker периодически запускает проверку внутри контейнера. При успешном коде завершения проверки статус остаётся healthy; при повторяющихся неудачах после предусмотренного числа попыток становится unhealthy. Механизм полезен для наблюдения за сервисом и систем оркестрации, поскольку позволяет отличать формально работающий процесс от доступного приложения.

5 ТЕГИРОВАНИЕ И ФИНАЛЬНАЯ ПРОВЕРКА

Финальному образу назначены теги 1.0.0, 20261019 и final. При многостадийной сборке дополнительно использован тег multistage. Равные идентификаторы на рисунке 9 показывают, что эти теги относятся к одному образу, а не к четырём независимым копиям. Тег позволяет выбрать нужную версию без повторной сборки.

Финальная проверка показывает версии и размеры образов, состояние контейнера и ответы API. На рисунке 9 видны дата 19.10.2026, сопоставление портов 5458:5458 и статус healthy. В ответах маршрутов отображаются приветствие владельцу, успешная проверка здоровья, сведения /me и флаг LAB2_SUCCESS_REKUN.

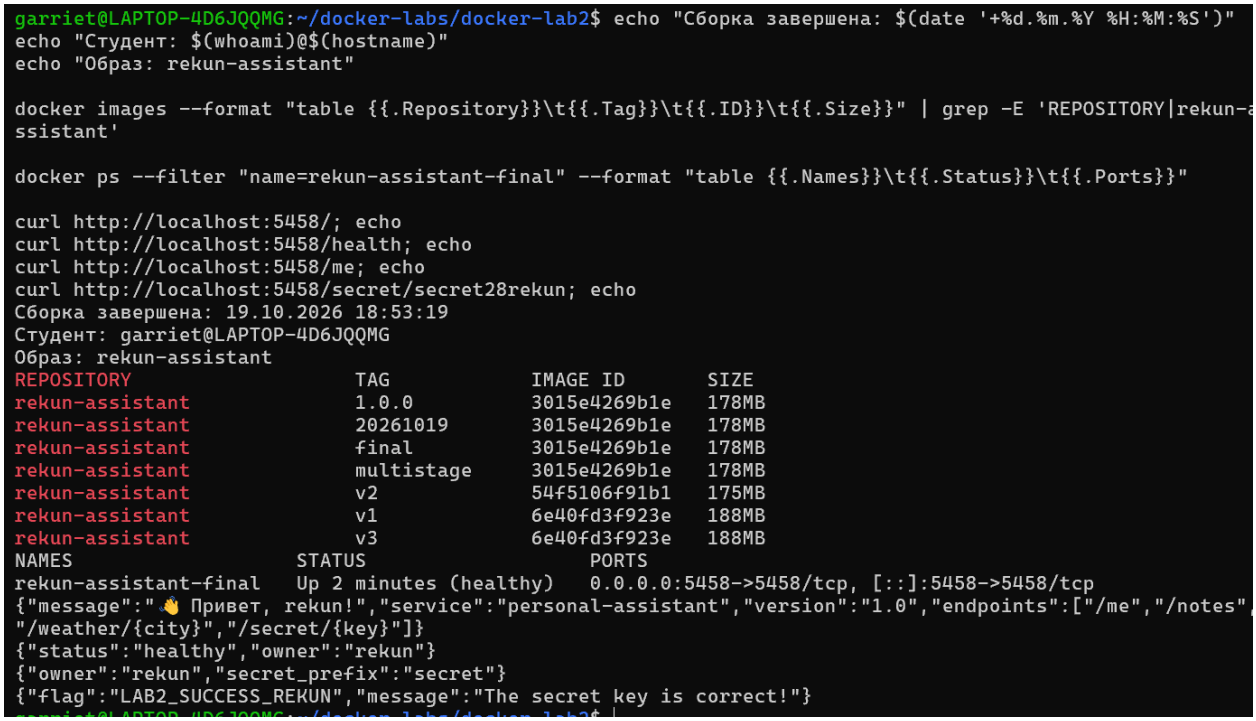


Рисунок 9 – Итоговая проверка приложения

Проверка подтвердила наличие требуемых тегов, использование персонального порта и ключа, а также работоспособность приложения.

Файлы проекта и README.md опубликованы в репозитории grettl/docker-lab2. Многостадийная сборка сохранена коммитом 688f733, а инструкция обновлена коммитом 2fd6c82. В README приведены команды для повторения сборки и запуска.

ЗАКЛЮЧЕНИЕ

В ходе работы был создан и запущен Docker-образ приложения Personal Assistant с персональным портом 5458. Работа приложения проверена HTTP-запросами: получены сведения о владельце, добавлены заметки и проверен секретный ключ.

После отключения кеша pip размер версии v2 уменьшился со 188 до 175 МБ. Дополнительно реализована многостадийная сборка; итоговый образ занимает 178 МБ. Настроены .dockerignore и Healthcheck, подтверждён статус healthy. Финальному образу назначены теги, а файлы проекта и инструкция запуска размещены на GitHub.